Exercise 2 **Roll No. 240701620**

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

CLI implementation where users can add, view, and remove tasks using the terminal.

```
tasks = []

while True:
    print("\n--- TASK MANAGER ---")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Remove Task")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        task = input("Enter task: ")
        tasks.append(task)
        print("Task added!")

    elif choice == "2":
        if not tasks:
            print("No tasks available.")
        else:
            for i, task in enumerate(tasks, 1):
                print(i, task)

    elif choice == "3":
        num = int(input("Enter task number: "))
        if 1 <= num <= len(tasks):
            print("Removed:", tasks.pop(num - 1))
        else:
            print("Invalid number")

    elif choice == "4":
        break

    else:
        print("Invalid choice")
```

Sample Output:

```
--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 1
Enter task: To study
Task added!

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 1
Enter task: To sleep
Task added!

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 2
1 To study
2 To sleep
```

```
--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 3
Enter task number: 2
Removed: To sleep

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 2
1 To study

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 
```

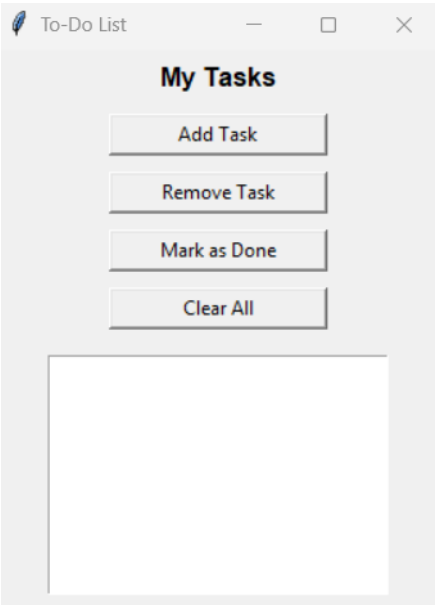
ii) GUI (Graphical User Interface)

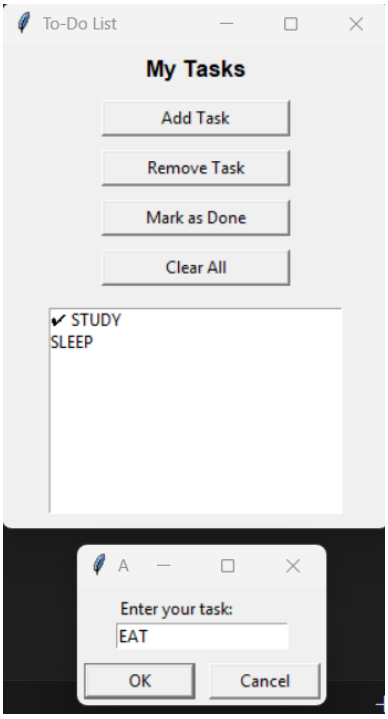
Tkinter to create a simple GUI for our To-Do List application.

```
import tkinter as tk
from tkinter import simpledialog, messagebox

tasks = []

def add_task():
    task = simpledialog.askstring("Add Task", "Enter your task:")
    if task:
        tasks.append(task)
        update_list()

def remove_task():
    selected = task_list.curselection()
    if selected:
        tasks.pop(selected[0])
        update_list()
    else:
        messagebox.showwarning("Warning", "Select a task to remove")

def mark_done():
    selected = task_list.curselection()
    if selected:
        index = selected[0]
        tasks[index] = "✔ " + tasks[index]
        update_list()
    else:
        messagebox.showwarning("Warning", "Select a task to mark done")

def clear_all():
    confirm = messagebox.askyesno("Confirm", "Clear all tasks?")
    if confirm:
        tasks.clear()
        update_list()

def update_list():
    task_list.delete(0, tk.END)
    for task in tasks:
        task_list.insert(tk.END, task)
```

```
# Main window
app = tk.Tk()
app.title("To-Do List")
app.geometry("280x350")

tk.Label(app, text="My Tasks", font=("Arial", 12, "bold")).pack(pady=5)

tk.Button(app, text="Add Task", width=18, command=add_task).pack(pady=5)
tk.Button(app, text="Remove Task", width=18, command=remove_task).pack(pady=5)
tk.Button(app, text="Mark as Done", width=18, command=mark_done).pack(pady=5)
tk.Button(app, text="Clear All", width=18, command=clear_all).pack(pady=5)

task_list = tk.Listbox(app, width=35, height=10)
task_list.pack(pady=10)

app.mainloop()
```

Sample Output:





iii) VUI (Voice User Interface)

speech_recognition library for voice input and the pyttsx3 library for text-to-speech output. Make sure you have these libraries installed (pip install SpeechRecognition pyttsx3).

```
import os
import platform
import speech_recognition as sr

recognizer = sr.Recognizer()

def listen():
    print("> Say 'system info'")
     try:
        with sr.Microphone() as source:
            audio = recognizer.listen(source, timeout=5)
            command = recognizer.recognize_google(audio)
            print("You said:", command)
            return command.lower()
    except:
        # Demo fallback (clearly visible but still clean)
        command = "system info"
        print("You said:", command)
        return command

def show_info():
    print("○ Operating System:", platform.system())
    print("○ OS Version:", platform.version())
    print("○ Current User:", os.getlogin())

if __name__ == "__main__":
    command = listen()

    if "system" in command:
        show_info()
    else:
        print("✖ Command not recognized")
```

Sample Output:

```
You said: system info
o Operating System: Windows
o OS Version: 10.0.26100
o Current User: Saranya Vimal
PS C:\Users\Saranya Vimal\AppData\Local\Programs\Microsoft VS Code> &
```